

LOCAL PDF WORKBENCH

Complete Project Plan for a Private, Offline-First Desktop PDF Application

A practical blueprint for building a personal PDF toolkit inspired by the attached I♥PDF-style dashboard. The application is designed to perform document operations locally, without uploading files or depending on cloud services.

Project type	Local desktop application
Primary objective	Private PDF processing on the user's own computer
Recommended stack	Tauri 2, React, TypeScript, Python, SQLite
Delivery approach	MVP first, then conversion, OCR, editing, local AI, and workflows
Planning horizon	Approximately 10–14 weeks for a solo developer, depending on scope

Prepared as a build-ready planning document

1. Executive Summary

The proposed Local PDF Workbench is a private desktop application that consolidates common PDF operations into one interface. Its user experience will resemble the attached reference: a searchable dashboard containing categorized tool cards, individual workspaces, progress feedback, and output management. Unlike a cloud PDF service, the application will be designed so that normal document processing takes place entirely on the local computer.

Core design principle: the application must remain useful with the internet disconnected. No feature should require a remote API, cloud storage, analytics service, or online conversion provider.

Primary outcomes

- A modern desktop dashboard containing PDF tools in one place.
- Local processing for PDF manipulation, conversion, OCR, and optional AI.
- No automatic upload of documents, extracted text, images, passwords, or generated files.
- A modular architecture in which each tool can be developed and tested independently.
- Safe output handling, validation, temporary-file cleanup, and clear error reporting.
- An extensible foundation for batch processing and multi-step workflows.

2. Project Scope

In scope

- Desktop application shell for macOS, Windows, and optionally Linux.
- Dashboard with categories, search, recent tools, and reusable tool cards.
- Local PDF manipulation and conversion engine.
- File picker, output-directory selection, preview, progress, cancellation, and job history.
- Optional local OCR, summarization, translation, and document search.
- Offline packaging, dependency management, and network-isolation validation.

Out of scope for the first release

- Cloud storage, user accounts, online collaboration, and remote processing.
- Online payment systems or subscription management.
- Automatic cloud fallback when a local operation fails.
- A promise of perfect PDF-to-Word or PDF-to-PowerPoint fidelity for every document.
- Password bypassing or unauthorized decryption of protected documents.

3. Recommended Technology Stack

Layer	Recommended technology	Purpose
Desktop shell	Tauri 2	Native desktop packaging, secure commands, window management, filesystem permissions.
Frontend	React + TypeScript	Dashboard, tool workspaces, previews, forms, progress, and settings.

Layer	Recommended technology	Purpose
Styling	Tailwind CSS	Consistent responsive layout and visual design.
Processing engine	Python	PDF manipulation, OCR orchestration, conversion, validation, and batch jobs.
PDF libraries	pypdf, PyMuPDF, pikepdf	Page operations, rendering, metadata, encryption, and object-level processing.
OCR	Tesseract OCR	Local text recognition for scanned documents.
Office conversion	LibreOffice	Local DOCX, XLSX, and PPTX conversion where supported.
Images	Pillow	Image conversion, resizing, compression, and JPG-to-PDF workflows.
Local persistence	SQLite	Preferences, job history, tool configuration, and local indexes.
Optional local AI	llama.cpp or another local runtime	Local summarization and extraction without cloud inference.

Why this architecture

Python has a mature document-processing ecosystem, while Tauri and React provide a modern desktop interface. The UI should never manipulate sensitive document contents directly when the operation can be performed by the local engine. The Tauri layer should validate commands, restrict filesystem access, and invoke only approved local processing functions.

4. System Architecture

Logical flow

Component	Responsibility
React/TypeScript UI	Displays tools, collects settings, selects files, shows progress, and presents results.
Tauri shell	Exposes controlled local commands, enforces permissions, and launches the engine.
Job manager	Creates job IDs, validates inputs, tracks progress, handles cancellation, and records status.
Python engine	Runs the selected PDF operation and returns structured success or error information.
Local libraries and executables	Perform PDF manipulation, rendering, OCR, and Office-compatible conversion.
Local filesystem	Stores input files, temporary files, generated outputs, logs, and optional models.
SQLite database	Stores preferences, job metadata, recent tools, and optional document indexes.

Required processing contract

Every tool should accept validated local input paths, tool-specific settings, and an output directory. It should return a structured result containing status, output paths, metadata, warnings, and error details. The original input should remain unchanged unless the user explicitly enables overwrite.

Example contract: Input: local file paths + settings + output directory Process: validate → execute → validate output → finalize Output: status + output path(s) + metadata + warnings/errors

5. Feature Roadmap

Stage	Tools and capabilities	Priority
MVP	Merge, split, rotate, organize pages, compress, watermark, page numbers, PDF-to-JPG, JPG-to-PDF, preview.	Must have
Phase 2	Edit/annotate, redact, crop, repair attempts, protect, password-based unlock, sign, compare, PDF/A, scan import.	High
Phase 3	PDF-to-Word, Word-to-PDF, PDF-to-Excel, Excel-to-PDF, PDF-to-PowerPoint, PowerPoint-to-PDF, HTML-to-PDF, forms, OCR, Markdown.	High
Advanced	Local summarizer, offline translation, workflow builder, batch processing, local document search, reusable presets.	Optional

Feature implementation notes

Feature	Implementation considerations
Merge / split / rotate / organize	Use pypdf for reliable page-level operations; validate page counts and output readability.
Compression	Combine object optimization with image downsampling; expose quality-versus-size settings.
Watermark / page numbers	Generate overlay pages with reportlab or use PDF page-merging capabilities.
Preview	Render pages locally using PyMuPDF; provide thumbnails and page navigation.
Redaction	Remove content permanently; do not merely draw a black rectangle over sensitive text.
OCR	Render pages, run Tesseract locally, and insert searchable text layers.
Office conversion	Use LibreOffice locally; clearly report layout limitations.
AI / translation	Bundle or separately install local models; never silently fall back to a cloud service.

6. User Interface Plan

Dashboard layout

- Top navigation: application name, search, settings, privacy status, and application version.
- Category filters: all tools, organize, optimize, convert, edit, security, intelligence, and workflows.
- Tool cards: icon, name, short description, favorite button, and recent-use indicator.
- Quick actions: open a file, open a folder, recent jobs, and create workflow.
- Privacy indicator: visibly state that processing is local and no cloud service is enabled.

Tool workspace layout

Area	Contents
Input panel	File picker, drag-and-drop area, file list, file validation, and page count.
Options panel	Tool-specific settings, presets, quality controls, page ranges, and output format.
Preview panel	Page thumbnails, rendered preview, selected pages, and before/after information.
Action bar	Start, cancel, reset, save preset, and choose output directory.
Result panel	Output filename, file size, warnings, validation status, and open-folder action.

7. Development Plan and Milestones

Phase	Estimated duration	Deliverables	Exit criteria
0. Planning	Week 1	Requirements, architecture, privacy model, tool inventory, repository.	MVP scope and design approved.
1. Desktop shell	Weeks 2–3	Tauri app, React dashboard, categories, search, settings, file picker.	Dashboard runs locally and navigates correctly.
2. Core engine	Weeks 4–6	Merge, split, rotate, organize, compress, watermark, page numbers, image conversion.	Core tools produce valid outputs and pass tests.
3. Advanced document tools	Weeks 7–9	Editing, redaction, security, OCR, Office conversion, Markdown, PDF/A.	Supported formats work with documented limitations.
4. Local intelligence	Weeks 10–11	Optional local AI, translation, workflows, batch processing, search.	Advanced features work without network access.
5. Hardening	Weeks 12–14	Security review, network tests, packaging, recovery, performance, documentation.	Offline installation and release checklist pass.

8. First Four Development Sprints

Sprint	Work items	Deliverable
Sprint 1	Initialize repository; configure Tauri, React, TypeScript, Python; create dashboard shell; define job contract.	Application launches and displays tool cards.
Sprint 2	Implement local file selection, output directory selection, Python engine runner, structured errors, and job status.	One end-to-end local job pipeline.
Sprint 3	Implement merge, split, rotate, and organize; add preview and page selection.	Reliable page-management toolkit.
Sprint 4	Implement compression, watermark, page numbers, JPG conversion, and automated tests.	Complete MVP processing set.

9. Privacy and Security Requirements

The phrase 'local application' must be treated as a testable engineering requirement. The application should be designed and verified so that document processing does not depend on the internet.

Control	Requirement
Network access	No remote API calls, analytics, advertising, telemetry, or cloud fallback. Test with network interfaces disabled.
Tauri permissions	Restrict filesystem and process permissions to only the paths and commands required by the application.
Local models	Store OCR, translation, and AI model files locally. Display model location and size.
Temporary files	Use an application-controlled temporary directory; remove temporary data after successful or cancelled jobs.
Sensitive logging	Never log document contents, extracted text, passwords, tokens, or full file data.
Output safety	Write to a temporary output, validate it, then atomically move it to the final destination.
Dependency control	Pin versions, retain lockfiles, document licenses, and verify bundled binaries.
Access control	Rely on OS account permissions and full-disk encryption for protection against other users.
Release verification	Monitor network traffic and test every MVP feature in an offline environment.

10. Local Data and File Management

Data	Storage location	Retention policy
Original documents	User-selected folders	Never modify by default.
Generated outputs	User-selected output folder	Retained until user deletes them.
Temporary files	App-controlled temp directory	Delete after job completion or cancellation.
Job metadata	SQLite database	Keep minimal metadata; provide clear-history action.
Settings	Local application configuration	Persist locally; export/import optional.
Logs	Local logs directory	Minimal diagnostics; configurable retention.
AI/OCR models	Local models directory	Managed by user; no remote model downloads during use.

11. Testing Strategy

Test corpus

- Text-based PDFs with selectable text.
- Scanned PDFs requiring OCR.
- Image-heavy PDFs and large multi-page documents.
- PDFs with tables, embedded fonts, annotations, forms, and metadata.
- Password-protected PDFs where the correct password is available.
- Malformed or damaged PDFs for error-handling and repair tests.
- Documents with unusual page sizes, rotations, and mixed orientations.

Test categories

Category	Tests
Unit tests	Validate page ranges, settings, filename handling, metadata, and error conditions.
Integration tests	Run each tool through the Tauri-to-Python job pipeline.
Output validation	Open generated PDFs with an independent parser and compare page counts and expected content.
Security tests	Run offline, monitor network connections, inspect logs, and verify no unexpected uploads.
Performance tests	Measure processing time and memory usage for small, medium, and large PDFs.
Recovery tests	Interrupt jobs, close the app, simulate low disk space, and verify cleanup.
UX tests	Check accessibility, keyboard navigation, readable errors, and clear progress feedback.

12. Risks and Mitigation

Risk	Impact	Mitigation
Complex PDF layouts convert poorly	High	Document limitations; preserve original; provide preview and warnings.

Risk	Impact	Mitigation
Large PDFs consume excessive RAM	Medium/High	Stream where possible, process page-by-page, cap concurrency.
OCR accuracy is low	Medium	Expose language settings, preview results, and allow reprocessing.
A dependency makes network requests	High	Audit dependencies, pin versions, run offline tests, and use firewall rules.
Temporary files remain after crashes	Medium	Use job directories, cleanup on startup, and clear retention rules.
Encryption or permissions are misunderstood	High	Require passwords where appropriate and avoid bypass claims.
Local AI is too resource-intensive	Medium	Make AI optional; show hardware requirements and model settings.
Cross-platform differences	Medium	Start with one OS, then add platform-specific packaging and tests.

13. MVP Definition of Done

- The application launches with internet access disabled.
- The dashboard supports search, categories, and tool navigation.
- Merge, split, rotate, organize, compress, watermark, page numbers, PDF-to-JPG, and JPG-to-PDF work.
- The user can select input files and an output directory through native dialogs.
- The original file is preserved by default.
- Every successful operation validates its output before reporting success.
- Errors are understandable and do not expose sensitive document contents.
- Temporary files are cleaned up after normal completion and cancellation.
- No external network connection is required by any MVP feature.
- Automated tests cover core operations and invalid-input behavior.
- A clean offline installation can be built and run on the target operating system.
- Documentation explains supported formats, limitations, privacy behavior, and troubleshooting.

14. Future Enhancements

- Batch processing of entire folders with configurable naming templates.
- Reusable presets such as 'compress for email' or 'prepare archival PDF'.
- Visual workflow builder: merge → compress → watermark → export.
- Local full-text document indexing and search.
- Local semantic search using an optional embedding model.
- Document comparison with visual overlays and text-diff views.
- Digital signatures using locally managed certificates.
- Plugin architecture for adding independent tools.
- Encrypted local workspace for especially sensitive documents.

- Portable mode for running from an external drive, subject to OS restrictions.

15. Immediate Next Steps

- 1 Choose the first target operating system. For a solo developer, start with one platform before expanding.
- 2 Create the repository and lock the initial technology versions.
- 3 Write the privacy policy for the application as an engineering specification, not merely marketing text.
- 4 Implement the Tauri + React dashboard and one reusable tool-card component.
- 5 Create the Python engine with a single merge operation and structured JSON-compatible job results.
- 6 Connect the dashboard to the engine through a controlled local command.
- 7 Add automated tests and a small non-sensitive PDF test corpus.
- 8 Complete the remaining MVP tools before starting complex editing, Office conversion, or AI features.
- 9 Perform offline network monitoring and packaging validation before calling the MVP complete.

Recommended starting point: build a reliable 8–10-tool MVP first. The most valuable early milestone is not the number of visible cards; it is a trustworthy local processing pipeline that preserves originals, validates outputs, handles errors, and works with the internet disconnected.

Appendix A — Suggested Repository Structure

```
local-pdf-workbench/ ■■■ apps/ ■ ■■■ desktop/ ■ ■■■ src/ ■ ■ ■■■ components/ ■ ■ ■■■  
pages/ ■ ■ ■■■ features/ ■ ■ ■ ■■■ merge/ ■ ■ ■ ■■■ split/ ■ ■ ■ ■■■ compress/ ■ ■ ■  
■ ■■■ organize/ ■ ■ ■ ■■■ convert/ ■ ■ ■■■ services/ ■ ■ ■■■ types/ ■ ■ ■■■ src-tauri/  
■ ■ ■■■ src/ ■ ■ ■■■ capabilities/ ■ ■■■ engine/ ■ ■■■ app/ ■ ■ ■■■ jobs/ ■ ■ ■■■ tools/ ■ ■  
■■■ converters/ ■ ■ ■■■ security/ ■ ■ ■■■ validation/ ■ ■■■ tests/ ■■■ assets/ ■■■ models/  
■■■ docs/ ■■■ scripts/ ■■■ pyproject.toml ■■■ package.json ■■■ README.md
```

Appendix B — Release Checklist

- Target OS build completed.
- Offline launch verified.
- MVP operations tested with representative PDFs.
- No unexpected network traffic observed.
- No sensitive information present in logs.
- Temporary-file cleanup verified.
- Output validation verified.
- Dependency versions and licenses documented.
- User documentation completed.
- Backup and recovery instructions documented.
- Installer or portable package tested on a clean environment.